Google Skills

Define your agent's identity 

This module builds on:

- Your working agent environment from module 1
- The default agent code created by `adk create`
- Understanding that
  model + tools + orchestration = agent

Understand your first agent

In module 1, you created a working agent using `adk create.`
When you opened `agent.py`, you saw this code:

```python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='A helpful assistant agent.',
    instruction='You are a helpful assistant.'
)
```

You verified it works with `adk web`. It's a success.

But what do these parameters actually mean? And how can you customize this agent for specific tasks? Think of this module as understanding the "Model" component from course 1, which is the reasoning brain of your agent.

The four core parameters

Reference: ADK docs – LLM Agent: Defining the Agent's Identity and Purpose

Every `LlmAgent` (also imported as `Agent`) is defined by four core parameters. Let's explore each one.

1. `model` (required)

What it is: The underlying large language model (LLM) that powers your agent's reasoning and decision-making.

Example:

```
Python
model='gemini-2.5-flash'
```

What this does:

- Determines your agent's intelligence and capabilities
- Affects cost per request and response speed
- Different models have different strengths (speed versus capability)

Available models:

- `gemini-2.5-flash` – Fast, efficient, and good for most tasks
- `gemini-2.5-pro` – More capable and better for complex reasoning
- See models documentation for the full list

Analogy: Like choosing the engine for your car, more powerful engines can handle more complex tasks but may cost more to run.

Important: This is a **required parameter**. Your agent cannot function without specifying a model.

2. `name` (required)

What it is: A unique string identifier for your agent.

Example:

```
Python
name='root_agent'
```

What this does:

- Identifies your agent internally within ADK
- Critical in multi-agent systems where agents refer to each other
- Used for logging, debugging, and agent delegation

Naming conventions:

✅ Use lowercase with underscores: `customer_support_agent`

✅ Be descriptive: `data_analysis_agent`, `math_tutor_agent`

❌ Avoid reserved names: `user`

❌ Don't use camelCase: Use `my_agent` not `myAgent`

From ADK docs:

"Every agent needs a unique string identifier. This name is crucial for internal operations, especially in multi-agent systems, where agents need to refer to or delegate tasks to each other."

Important: This is a required parameter. Every agent must have a name.

3. `description` (optional, recommended for multi-agent)

What it is: A concise summary of what your agent does.

Example:

```python
description='Answers user questions about the capital city of a given country.'
```

What this does:

- Used by other agents to decide if they should route tasks to this agent
- Helps in multi-agent systems where agents delegate to each other
- Not used by the agent itself for its own behavior

From ADK docs:

"This description is primarily used by other LLM agents to determine if they should route a task to this agent. Make it specific enough to differentiate it from peers."

When to use:

✅ Building multi-agent systems with delegation

✅ Agents that will be called by other agents

⚠️ Less important for single-agent applications

Good descriptions:

✅ "Handles customer billing inquiries and processes payment updates"

✅ "Analyzes sales data and generates weekly performance reports"

✅ "Helps students learn algebra by guiding them through problem-solving steps"

❌ "Billing agent" (too vague)

❌ "Helper" (not specific enough)

4. `instruction` (critical but optional)

What it is: The behavioral blueprint that guides how your agent acts and responds.

Example:

```python
instruction="""You are a helpful assistant that tells the current time in cities.
Use the 'get_current_time' tool for this purpose."""
```

What this does:

- Defines the agent's personality and communication style
- Specifies the agent's core task or goal
- Sets boundaries and constraints on behavior
- Guides when and how to use tools (covered in module 3)
- Shapes the output format

From ADK docs:

"The instruction parameter is arguably the most critical for shaping an LlmAgent's behavior. It tells the agent its core task or goal, its personality or persona, constraints on its behavior, and how and when to use its tools."

Tips for effective instructions (from ADK docs):

- Be clear and specific: avoid ambiguity, clearly state the desired actions and outcomes
- Use markdown: Improve readability for complex instructions using headings, lists, etc.
- Provide examples (few-shot): For complex tasks or specific output formats, include examples
- Guide tool use: don't just list tools, explain when and why the agent should use them

`description` versus `instruction` – The key difference

This is critical to understand:

Parameter	Audience	Purpose	Example
`description`	**Other agents**	“Should I route this task here?”	“Handles billing inquiries”
`instruction`	**This agent**	“How should I behave?”	“You are a billing expert who...”

Concrete example:

```python
# Multi-agent system with 3 agents
billing_agent = Agent(
    model='gemini-2.5-flash',
    name='billing_agent',
    # OTHER agents read this to decide if they should delegate here
    description='Handles customer billing inquiries and payment processing',
    # THIS agent reads this to know how to behave
    instruction="""You are a billing specialist.

    When helping customers:
    1. Be empathetic and patient
    2. Explain charges clearly
    3. Use the billing_lookup tool to check account details
    4. Never promise refunds without manager approval

    Always maintain a professional, helpful tone."""
)

support_agent = Agent(
    model='gemini-2.5-flash',
    name='support_agent',
    description='Handles general customer support questions',
    instruction="""You are a customer support agent.

    If a question is about billing, transfer to the billing_agent.
    Otherwise, help the customer directly."""
)
```

In this example:

- `support_agent` reads `billing_agent`'s **description** to decide: “Is this a billing question?”
- `billing_agent` reads its own **instruction** to know: “How do I handle billing questions?”

The `root_agent` naming convention

Reference: Python Quickstart for ADK

You might have noticed your agent variable is named `root_agent`:

```python
root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',  # Internal name (inside Agent)
    # ...
)
```

Why root_agent?

ADK command-line tools look for a Python variable named root_agent as the entry point to your agent system. This is a convention that allows ADK to discover and run your agent.

From ADK docs:

“The agent.py file contains a root_agent definition, which is the only required element of an ADK agent.”

Can the internal name be different from the variable name?

Yes. The `name` parameter inside `Agent()` is separate from the variable name:

```python
# Internal ADK name (used for logging, delegation)
my_specialized_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',  # Used by ADK internally
    description='Helps students with algebra',
    instruction='You are a patient math tutor...'
)

# Variable name that ADK tools look for (must be root_agent)
root_agent = my_specialized_agent
```

Key rule: Always assign your main agent to a variable named `root_agent`, so ADK tools can find it.

In module 3, you’ll learn how ADK tools use root_agent to run your agent in different ways.

Writing effective instructions

Reference: ADK docs – Guiding the Agent: Instructions

Instructions guide your agent's behavior and personality.
For this course, we'll keep them simple to help you understand the basics.

Simple approach for course 2:

```python
instruction="You are a patient math tutor. Help students with algebra problems."
```

This basic instruction is enough to get started. It tells the agent:

- Its role (math tutor)
- Its personality trait (patient)
- Its task (help with algebra)

Do you want to learn professional instruction patterns? In course 3: Crafting Better Agent Behavior, you'll learn advanced techniques, including:

- Multi-section instruction structures
- Persona and boundary definitions
- Few-shot examples
- Production-ready instruction patterns

For now, let's focus on understanding how basic instructions work.